

A Confidence-Gated Hybrid Perceptron Branch Predictor for an RV32I Pipeline

RTL design, verification, and FPGA implementation study on Artix-7

Prasit Mazumder

Project report

Summary. I built a perceptron-based dynamic branch predictor into a 5-stage RV32I pipeline, verified it against three conventional predictors, and took it through Vivado synthesis and place-and-route for an Artix-7 xc7a35t target. On a twelve-kernel branch benchmark the perceptron has a **6.8 %** misprediction rate over control transfers, against gshare’s 13.5 % and bimodal’s 11.5 % — 40 % fewer mispredictions than the best conventional predictor I implemented. It also costs 63 % of the core’s LUTs and drops F_{max} from 96.3 MHz to 86.1 MHz, so on this benchmark and this device *gshare still has the lower estimated execution time at the achieved F_{max}* . This report covers the design, the verification, and that trade-off.

1 Contribution and key results

What I implemented

I implemented a perceptron-based dynamic branch predictor in synthesisable SystemVerilog and integrated it into a 5-stage RV32I pipeline. On top of the published perceptron algorithm I implemented speculative global-history tracking with exact repair, adaptive control of the training threshold, confidence-gated hybrid prediction against the BTB’s existing counter, and a two-level overriding organisation with a pipelined adder tree. I built a common predictor interface so that bimodal, gshare and perceptron configurations could be evaluated under otherwise identical pipeline conditions, developed directed and system-level verification, cross-checked every result across Icarus Verilog and Vivado xsim, and carried out synthesis, place-and-route and parameter sweeps to characterise the accuracy, area and timing trade-offs.

The perceptron algorithm itself is Jiménez and Lin’s, and the four extensions are based on published techniques. Section 5 states exactly which parts come from the literature and which are my own design decisions.

Key results

	gshare	perceptron	source
Misprediction rate	13.5 %	6.8 %	simulation
Benchmark cycles	13 842	13 571	simulation
Slice LUTs (core + predictor)	2 277	4 412	place-and-route
F_{max}	96.3 MHz	86.1 MHz	static timing
Estimated runtime at F_{max}	143.7 μs	157.6 μ s	derived

The perceptron predicts better and executes in fewer cycles, but costs enough area and frequency that gshare completes the benchmark sooner. The full four-way comparison, including bimodal and a no-predictor baseline, is in Section 14.

2 Introduction

This project adds a perceptron-based dynamic branch predictor to a 5-stage RV32I processor, compares it against three conventional predictors under identical conditions, and carries it through to a placed-and-routed implementation on a Xilinx Artix-7 FPGA.

A trace-only accuracy comparison cannot capture what a predictor does to the clock frequency of the processor it is attached to, and a predictor that saves cycles while lowering the clock may leave the machine slower overall. This project therefore measures both from the same RTL: cycle counts from simulation of real compiled code, and F_{max} and area from synthesis and implementation of that same design.

Section 3 covers the background a reader may want before the rest makes sense — what a control hazard is, what a branch predictor does about it, and how the conventional schemes work. Readers familiar with that material can start at Section 4.

3 Background

3.1 Pipelining and the control hazard

A pipelined processor overlaps the execution of several instructions. The design used here has five stages — fetch (F), decode (D), execute (E), memory (M) and writeback (W) — so up to five instructions are in flight at once, and in the ideal case one instruction completes every cycle.

That overlap assumes the processor knows which instruction to fetch next. For ordinary arithmetic it does: the next instruction sits at the next address. A conditional branch breaks the assumption. Consider `beq x1, x2, target`: whether the next instruction comes from `target` or from the following address depends on comparing two registers, and that comparison happens in the execute stage. Fetch needs the answer two stages before it exists.

This is a *control hazard*. The simplest response is to stall fetch until the branch resolves, which on this pipeline wastes three cycles on every branch. The usual response is to guess, continue fetching, and undo the damage if the guess was wrong:

cycle	1	2	3	4	5	6	7
beq (branch)	F	D	E	M	W		
next instruction (guessed)		F	D	<i>killed</i>			
next instruction (guessed)			F	<i>killed</i>			
next instruction (guessed)				<i>killed</i>			
instruction at correct target					F	D	E

If the guess is right, nothing is wasted and the branch costs nothing. If it is wrong, the three instructions fetched on the wrong path are discarded and the branch costs three cycles. So the average penalty is

$$\text{cycles lost per branch} = 3 \times (\text{misprediction rate}),$$

and on this benchmark, where 3 572 of 12 457 instructions are control transfers, the difference between a 70 % and a 7 % misprediction rate is worth roughly a third of total runtime.

3.2 What a branch predictor has to supply

To keep fetching without stalling, the processor needs two things at fetch time, before the instruction has even been decoded:

- **Is this instruction a branch, and where does it go?** Supplied by a *branch target buffer* (BTB) — a small cache indexed by the instruction address, holding the target address of branches seen before.
- **If it is a conditional branch, will it be taken?** This is *direction prediction*, and it is what distinguishes the schemes compared in this report.

Every predictor in this project shares the same BTB and differs only in the direction predictor. A BTB miss is not a misprediction in the usual sense — the processor simply does not know the instruction is a branch yet — but it costs the same three cycles, so the BTB has to be adequately sized before direction prediction can be measured at all.

3.3 Conventional direction predictors

Bimodal. The oldest practical scheme. A table of 2-bit saturating counters is indexed by the low bits of the branch address. Each counter moves one step towards *taken* when its branch is taken and one step towards *not taken* otherwise, and the prediction is whichever side it currently leans. Two bits rather than one gives hysteresis: a loop branch that is taken ninety-nine times and not taken once mispredicts only on the exit, rather than on the exit and again on the next entry.

Bimodal learns per-branch bias and nothing else. A branch that alternates taken, not taken, taken, not taken, defeats it completely — the counter oscillates in the middle and is wrong every time.

Gshare. Adds correlation. A *global history register* shifts in one bit per branch recording whether it was taken, and gshare indexes its counter table with the branch address XOR-ed with that history. The same branch under different recent histories therefore lands on different counters and can learn different behaviour for each. This captures patterns bimodal cannot, including alternation and short repeating sequences.

The weakness is capacity. To learn a pattern spanning h branches, the table must hold a counter for every history pattern that matters, and the number of distinct patterns grows as 2^h . Doubling the useful history length doubles the table. In practice gshare is limited to roughly 10–15 history bits before the table becomes impractical, and long before that, unrelated branches begin colliding on the same counter — *aliasing* — and interfere with each other’s training.

3.4 The perceptron approach

Jiménez and Lin [1] proposed replacing the counter table with a small linear classifier for each branch. Instead of one counter per history *pattern*, there is one signed weight per history *bit*:

$$y = w_0 + \sum_{i=1}^h x_i w_i, \quad x_i = \begin{cases} +1 & \text{if the } i\text{th previous branch was taken} \\ -1 & \text{otherwise} \end{cases}$$

and the branch is predicted taken when $y \geq 0$. The weight w_i records how strongly, and in which direction, the outcome i branches ago correlates with this branch. The bias w_0 records the branch’s own tendency, which is what bimodal would have learned on its own.

The consequence is the central argument for the approach: storage grows *linearly* in h rather than exponentially, so a perceptron can consider far longer histories than a counter table of comparable size. The costs are that a dot product is more work than a table lookup, and that a linear classifier can only separate linearly-separable functions — it cannot learn XOR of two history bits, which a counter table represents without difficulty.

Training is equally simple. After the branch resolves with outcome $t \in \{-1, +1\}$, each weight is nudged by $w_i += t x_i$, which increases weights that agreed with the outcome and decreases the rest. Training happens when the prediction was wrong, or when it was right but $|y|$ was below a threshold θ — the second condition stops weights saturating on easy branches and leaving the predictor unable to adapt.

4 Starting point, and what was wrong with it

I began from two things I already had: a working RV32I pipeline without branch prediction, and an earlier attempt of mine at bolting a perceptron onto it that produced plausible-looking waveforms and completely useless predictions.

Working out why took longer than rewriting it. The defects were structural rather than typographical, and several masked each other:

- A three-cycle state machine drove the weight-table read, but its request valid signal was tied high, so the training-read state was never reached. The weights were read, never written, and stayed at zero for the entire run. A perceptron whose weights are all zero outputs $y = 0$ forever, which the

sign test resolves to a constant prediction — so the thing *looked* like it was working, at roughly the accuracy of a static predictor.

- The dot product was accumulated with non-blocking assignments inside a loop. Every iteration read the same pre-loop value of the accumulator, so the sum collapsed to its last term.
- Two separate `always_ff` blocks drove the global history register and the pattern table.
- The history “shift” was written as `{ghr[W-1:1], exTaken}`, which rewrites bit 0 and shifts nothing.
- Training used the history as it stood at training time rather than the history that was live when the prediction was made.

I rewrote the predictor rather than patching it, because with this many independent faults present at once, each fix was masked by the others and could not be verified on its own.

Two further problems were in the surrounding system rather than the predictor, and both had been quietly corrupting every measurement. The CSR file had no reset, so the cycle and instruction counters powered up as `x` in simulation and the software divide routine that formats them never terminated. And the linker script never cleared `.bss`, so static state started as whatever the memory model happened to contain. Neither is a branch-prediction bug. Both had to be fixed before any branch-prediction number meant anything.

5 Relationship to published work

The predictor draws on several published techniques, and this section sets out which parts come from where.

The perceptron predictor is Jiménez and Lin’s [1]. I implemented it as described: one weight vector per branch, dot product against the global history mapped to ± 1 , prediction from the sign, and training on either a misprediction or an insufficient margin. Four extensions sit on top of it, and the concepts behind all four are also in the literature. What is mine is the combination on this core, some specific implementation choices, and the measurement.

Element	Where the idea comes from	What I did
Perceptron predictor	Jiménez & Lin, HPCA-7 2001 [1]	Implemented as published. No claim of novelty.
N1 — speculative global history with repair	Standard practice in speculative pipelines	Exact repair: a committed copy is kept alongside the speculative one and restored bit-accurately on a misprediction, including the spurious bit left by a BTB false hit.
N2 — adaptive training threshold	Seznec, ISCA 2005 [4]	Simplified to a single counter per predictor rather than per-table, sized for this pipeline.
N3 — confidence gate	Hybrid/tournament predictors, McFarling 1993 [3]	The gate consults the BTB’s <i>existing</i> 2-bit counter instead of adding a separate chooser table — so it costs no new storage. This is the main design choice specific to this work.
N4 — overriding organisation	Jiménez, Keckler & Lin, MICRO-33 2000 [2]	The idea is published; the work here is its implementation in this pipeline — the pipelined adder tree, the decode-stage shadow state and the flush plumbing it needs — and the measurement of what it buys in F_{max} against what it costs in cycles.

In short, these are established techniques brought together on one core, implemented in synthesisable RTL and evaluated for area and timing through synthesis and place-and-route against a real device, rather than assessed for accuracy alone in a trace simulator.

6 The baseline machine

The host is a 5-stage RV32I pipeline — fetch, decode, execute, memory, writeback — with full forwarding and a load-use interlock. Branches resolve in execute, so a misprediction costs three cycles.

Every predictor plugs into the same interface. The pipeline, the branch target buffer, the unconditional-jump handling, the speculative-history machinery and the verification framework are held fixed across all four configurations; the direction-prediction mechanism — and, for the perceptron, the override and pipelining machinery that mechanism requires — are the intentional differences. The perceptron is therefore not a drop-in swap for a counter table, and its override logic is part of what is being compared, not a confound to be subtracted out.

6.1 Performance counters, and exactly what they count

Four custom CSRs let the benchmark measure itself from inside the machine rather than relying on the testbench. Since every number in this report derives from them, it is worth stating precisely what each one counts in the RTL:

CSR	Driven by	Counts
0x80	wrongBranch	Execute-stage mispredictions, as <code>pcSel ^ bPredictedTaken</code> — the resolved direction disagreeing with the predicted one.
0x81	controlXfer	<code>jump branch</code> in execute, so <i>conditional branches plus unconditional jumps</i> , whether taken or not.
0x82	validInst	Cycles in which the execute stage holds a non-zero PC.
0x83	—	Cycles.

Two consequences follow, and I state them because the alternative readings are more flattering than the truth.

First, 0x82 is an execute-stage validity count, not a formal retirement event. In this pipeline the two coincide: it is in-order, has no exceptions or aborts, and mispredicted instructions are killed in fetch and decode so they never reach execute. But the signal is `lpc`, which means an instruction at address 0x00000000 would not be counted. Throughout this report “instructions” means this counter, and CPI means cycles per counted instruction.

Second, the misprediction *rate* quoted for the four-way comparison is mispredictions divided by control transfers, and control transfers include unconditional jumps, which are never mispredicted once the BTB has seen them. That makes the rate slightly optimistic in absolute terms. It is applied identically to all four configurations, so the comparison between them is unaffected — but where the perceptron’s internal statistics are quoted (Section 11) those are conditional-branch-only and the two figures should not be mixed.

Figure 2 shows the same structure as the synthesis tool sees it.

7 RTL organisation

The design is about 2 800 lines of SystemVerilog, organised so that the four predictor configurations can be swapped without touching the pipeline.

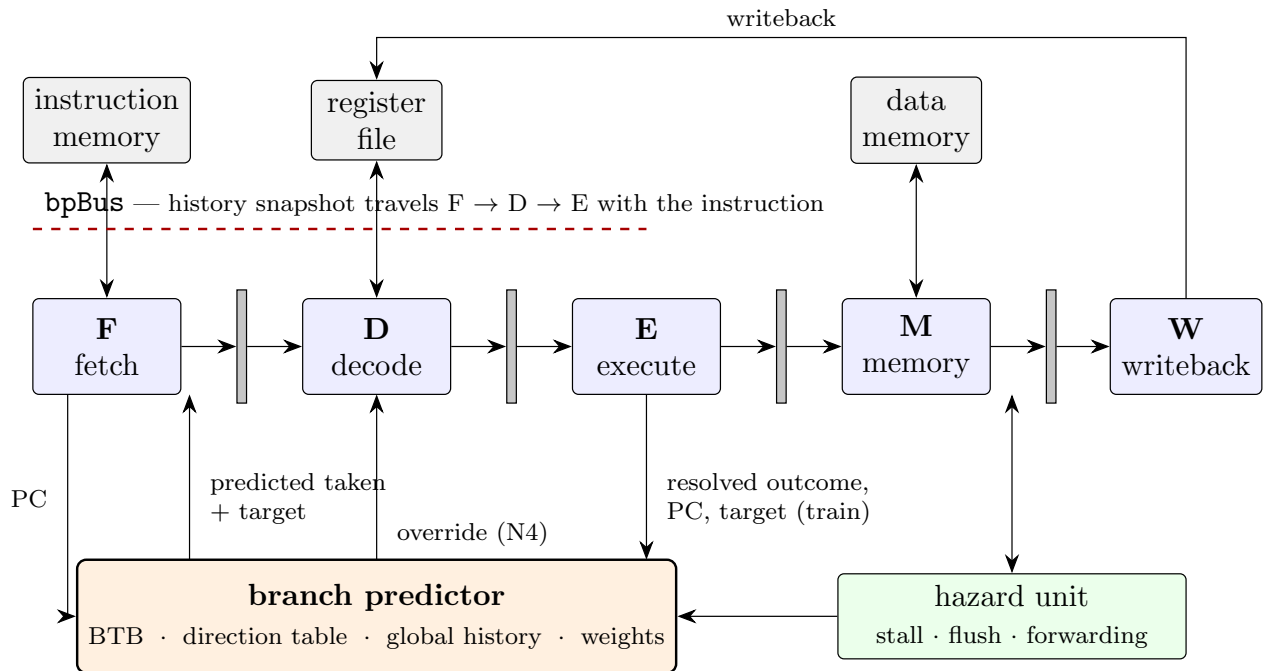


Figure 1: Architecture. The predictor attaches at three points: it reads the PC and returns a prediction in fetch, it can overrule that prediction one cycle later in decode (N4), and it is trained from the resolved outcome in execute. The dashed line is the history snapshot that travels with the instruction so that training uses the same inputs the prediction was made from.

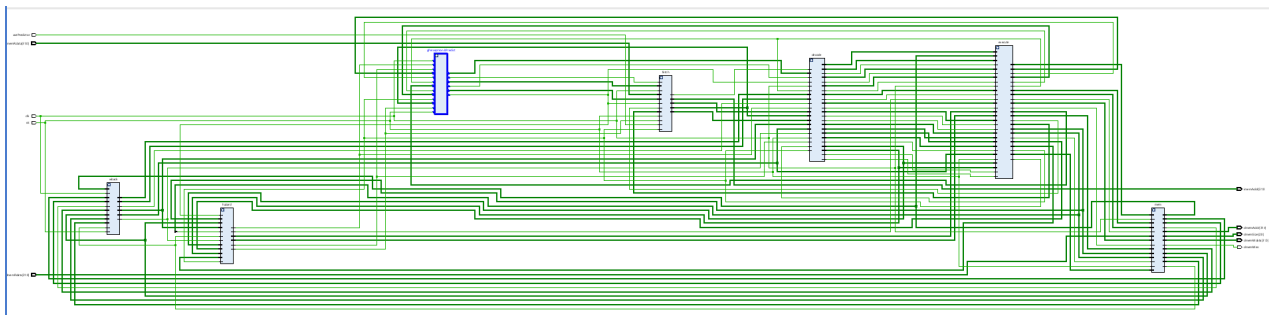


Figure 2: Elaborated design, top level, in Vivado — the tool’s view of Figure 1, before any technology mapping. Included to show that the block diagram corresponds to real RTL rather than to an intention; the detail is not meant to be read.

File	Role	Lines
perceptronPredictor.sv	the predictor, N1–N4	718
core.sv	pipeline assembly, predictor selection	340
classicPredictor.sv	bimodal and gshare reference	246
stageExecute.sv	ALU, branch resolution, forwarding	218
controlUnit.sv	instruction decode	132
stageDecode.sv	register read, history bus	131
bp_config.vh	every tunable parameter	98
hazardUnit.sv	stall and flush generation	72
TB.sv, TB_predictor.sv	system and standalone testbenches	459

7.1 One interface, four predictors

Every predictor — perceptron, gshare, bimodal — presents the same port list, so `core.sv` instantiates whichever one a compile-time define selects and nothing else in the pipeline changes. The ports fall into three groups, one per pipeline stage that the predictor touches:

- **Fetch.** `fetchPc` in; `fetchHit`, `fetchTarget` and `fetchBpBus` out. The last of these is the point of the whole scheme: it is a snapshot of the global history *as it stood when this prediction was made*.
- **Decode.** `ovrValid`, `ovrTaken`, `ovrTarget` out — the N4 override. Also `dStall` and `dFlush` in, so the predictor’s decode-stage shadow state stays in step with the real pipeline register. Forgetting to drive `dFlush` is precisely the bug the standalone testbench caught.
- **Execute.** `exBranch`, `exTaken`, `exWrong`, `exPc`, `exTarget`, and `exBpBus` — the returning history snapshot.

That `bpBus` rides the pipeline registers alongside the instruction, `fetch` → `decode` → `execute`. It carries the 24 history bits plus one flag saying whether this fetch actually advanced the speculative history. When the branch resolves, training uses the history that produced the prediction rather than the history that happens to be current — which was one of the five bugs in my first attempt, and the one whose symptom was least obvious.

Holding that interface fixed is what makes the comparison in Section 14 mean anything. The BTB, the jump handling, the speculative-history machinery and the pipeline around them are identical in all four configurations, so a measured difference cannot be attributed to some incidental change in the core. What differs is the direction-prediction mechanism together with whatever that mechanism needs in order to work — for the perceptron, the decode-stage override and the pipelined adder tree. Those are properly part of its cost, which is why the area and F_{max} figures in Section 13 are quoted for the predictor as a whole rather than for a weight table in isolation.

7.2 Parameters

All geometry lives in `bp_config.vh`, and every parameter is wrapped in an ‘`ifndef`’ guard so that a `-D` on the command line wins. That guard is not cosmetic. Without it the header silently overrides the command line, and a parameter sweep runs happily to completion while producing identical numbers for every point — a failure that looks exactly like “this parameter does not matter”. Every result in Section 12 depends on these guards being present.

8 The predictor as built

8.1 Geometry

The predictor holds 128 weight vectors. Each vector is 25 signed 8-bit weights — one bias plus one per history bit, with a 24-bit global history. Total storage is $128 \times 25 \times 8 = 25\,600$ bits, or 3.125 KiB. The reference predictors use an 8192-entry table of 2-bit counters, which is 16 Kbit, plus the same 256-entry BTB.

The storage is not matched, and that is a limitation. At 25.6 Kbit against 16 Kbit the perceptron gets about $1.6\times$ the direction-prediction storage, so part of its accuracy advantage could in principle be bought rather than earned. A 2-bit counter table can only be sized in powers of two, and neither 8192 entries (16 Kbit) nor 16 384 (32 Kbit) matches 25.6 Kbit; I took the smaller.

The sweeps in Section 12 argue against the advantage being a storage artefact, without settling it. Accuracy is unchanged at 12 history bits (244 mispredictions) and unchanged again at 5-bit weights (243), each measured with the other parameters at their defaults. A perceptron at 12 bits of history and 5-bit weights would need $128 \times 13 \times 5 = 8\,320$ bits — half the reference predictors’ budget — and on the evidence of those two sweeps would be no less accurate. That combined configuration was not itself measured, so this is an indication rather than a result, and the honest position is that the comparison in this report is between a 25.6 Kbit perceptron and a 16 Kbit counter table.

8.2 Prediction and training

The mechanism is the one described in Section 3: the branch address selects a row, each history bit steers the sign of its weight, and the prediction is the sign of the sum. Two implementation details are worth recording.

The history bits are never used as part of the index. They select signs, which is why the table

is addressed by branch address alone and why 128 rows are enough for a benchmark with 117 distinct branch sites. And the multiplication in $x_i w_i$ is not a multiplier: since $x_i \in \{-1, +1\}$, it is a conditional two's-complement negation, which on an FPGA is an XOR with the history bit feeding the carry-in of the adder that consumes it.

θ is initialised to the value the original paper prescribes, $1.93h + 14$, which for $h = 24$ gives 60. From there it adapts (N2).

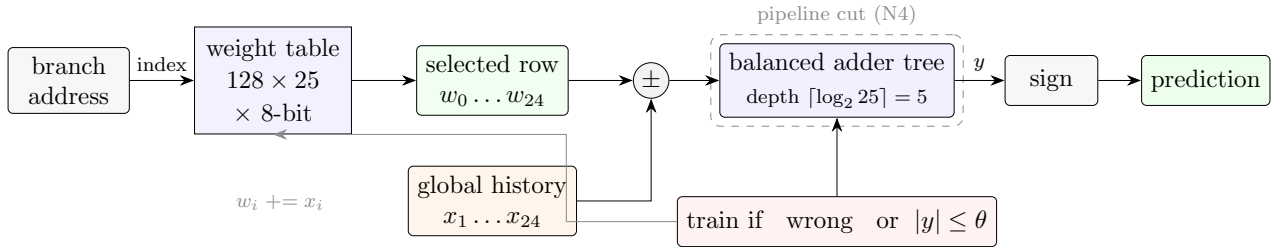


Figure 3: Prediction datapath. The history bits do not index the table — they select the sign of each weight. This is why storage grows linearly with history length rather than exponentially, which is the central argument for the approach.

8.3 Why a balanced adder tree

My first version accumulated the 25 terms in a sequential chain. It was correct and it was slow: a 25-deep adder chain in the fetch path held the whole core to 52 MHz. Restructuring it as a balanced tree cuts the depth from 25 to $\lceil \log_2 25 \rceil = 5$. The arithmetic is identical — integer addition is associative, and I confirmed the misprediction count was unchanged at 246 before and after — but F_{max} moved to 55.9 MHz on its own, and the tree is what made the later pipelining possible at all.

9 The four extensions

9.1 N1 — speculative history with exact repair

A predictor that trains on 24 bits of global history needs those bits to be current at fetch time. But the true outcome of a branch is not known until it reaches execute, three stages later. If the history register is only updated on resolution, then by the time a branch is predicted the last three branches are missing from its history, and the correlation the predictor exists to exploit is exactly what has been lost.

So the history is updated speculatively at fetch, with the predicted outcome, and a committed copy is maintained separately at execute. On a misprediction the speculative copy is restored from the committed one with the real outcome appended.

One case here caused a bug that took a while to find. If the BTB reports a hit on an address that is not actually a branch — a stale entry — fetch will have shifted a speculative bit into the history for an instruction that will never resolve. The repair has to drop that bit too, not merely restore the committed value.

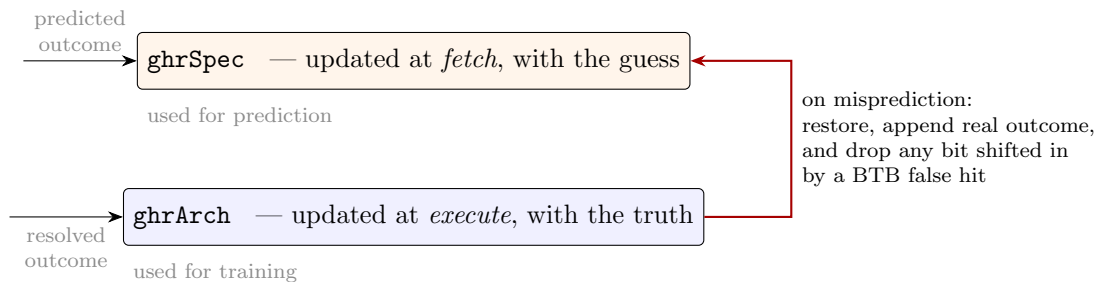


Figure 4: N1. Two history registers. The speculative one is fast but may be wrong; the committed one is always right but three cycles late. Repair moves state from the second to the first.

9.2 N2 — adaptive training threshold

The threshold θ decides when a correct-but-marginal prediction still triggers training. Jiménez and Lin fix it as a function of history length. A fixed value has to be a compromise across branch behaviours that vary widely, so here θ is a counter that rises when the predictor mispredicts too often and falls when it is comfortably accurate. The idea is Seznec’s; the implementation is a single counter for the whole predictor rather than one per table, which is all this predictor’s size justifies.

It is observable in simulation: θ starts at 60, and by the end of the benchmark has settled at 44. The predictor decided it could afford to train less aggressively than it started out assuming.

9.3 N3 — confidence gate

A perceptron is weakest early, when its weights are near zero and $|y|$ is small. A 2-bit counter is at its best exactly there — a strongly-biased counter on a consistently-taken branch is hard to beat. So when $|y|$ is small *and* the BTB’s counter for that branch is in a strongly-biased state, the counter’s answer is used instead.

The gate reads the BTB’s existing counter rather than a dedicated one. A conventional hybrid predictor adds a chooser table, which is extra storage and another structure that has to be trained before it is useful. Here the counter is already present for target prediction, so the gate adds only comparison logic.

That works because of how I encoded the 2-bit counter. The four states are assigned so that the two questions the gate needs to ask fall out as single bit tests:

State	Encoding	bit 0 — direction	bit 1 — strength
strongly taken	10	0 (taken)	1 (strong)
weakly taken	00	0 (taken)	0 (weak)
weakly not taken	01	1 (not taken)	0 (weak)
strongly not taken	11	1 (not taken)	1 (strong)

So the counter’s prediction is $\sim\text{count}[0]$ and its confidence is $\text{count}[1]$ — direction and confidence read out of the same two bits with no extra storage and no extra table. The gate is then one expression:

$$\text{useBimodal} = (|y| \leq \Theta_{\text{gate}}) \wedge \text{count}[1], \quad \Theta_{\text{gate}} = 16.$$

A worked case. Suppose a branch has been taken on the last forty executions. Its BTB counter is at 10, strongly taken. The perceptron has also learned it, so y is large and positive — say +79, a value visible in Figure 9 — and $|y| = 79 > 16$, so the gate does not fire and the perceptron’s own answer is used. Both agree anyway. Now suppose the branch is newly encountered: the perceptron’s row is still near zero, $y = -1$, and the sign test would predict not taken on essentially no evidence. If the counter has meanwhile reached 10, the gate fires and the prediction becomes taken. The perceptron still trains on the outcome; it simply does not get to vote while it has nothing to say.

The same expression is evaluated at three points — in fetch, in the decode override, and again during training — against the y belonging to that stage. Using one expression three times rather than three variants is what keeps the recorded prediction consistent with the one actually acted on, which matters because the training step asks “was the prediction wrong?” and would otherwise sometimes be answering about a different prediction.

Section 12 shows this is not free to tune: the gate has a clear optimum, and setting it too wide is worse than not having it at all.

9.4 N4 — overriding predictor with pipelined training

This is the extension that matters most, and it exists for a timing reason rather than an accuracy one.

If the perceptron has to produce its answer in time to steer the next fetch, the critical path runs address $\rightarrow$ weight RAM $\rightarrow$ adder tree $\rightarrow$ sign $\rightarrow$ next-PC multiplexer $\rightarrow$ PC. Measured on this

device, that path is 17.6 ns and caps the core at about 56 MHz — and the path has nothing to do with how good the prediction is.

So fetch is steered by the BTB’s 2-bit counter, which is a short path, while the perceptron computes across two cycles and delivers its verdict one cycle later. If it disagrees it redirects fetch. The cost is one bubble rather than the three a full misprediction would cost.

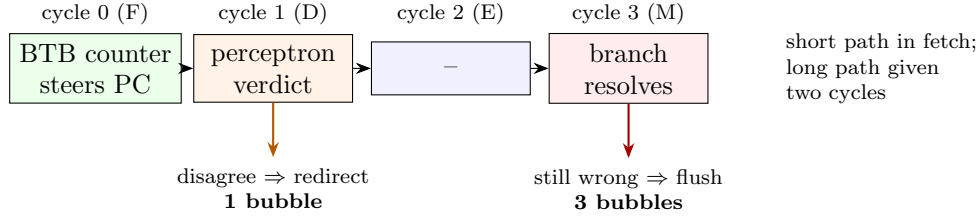


Figure 5: N4. The fast predictor never leaves the fetch critical path. The perceptron overrules it one cycle later for the price of a single bubble.

Adding the override alone moved F_{max} by only 0.4 MHz, which was initially disappointing. The reason was that two paths were nearly tied: the fetch path and the *training* path, which recomputes the dot product to decide whether to update. Cutting only one left the other exposed. Pipelining the training recomputation as well is what produced the real gain.

	cycles	F_{max} (MHz)	est. runtime (μ s)
sequential accumulator	13 364	52.0	257.0
balanced adder tree	13 364	55.9	239.1
+ N4 override	13 571	56.3	241.0
+ pipelined training	13 571	86.1	157.6

Misprediction count is 246 in all four rows — these are timing changes, not accuracy changes. The override costs 207 cycles (the bubbles) and buys 30 MHz, which is a clear net win.

10 How I verified it

A branch predictor can produce plausible statistics while being wrong, so the design was checked three separate ways.

Architectural transparency. A branch predictor must not change what a program computes — only how fast it computes it. The benchmark ends by printing a checksum. All four predictor configurations print `Final sink: 801`, and all four count exactly 12 457 instructions and 3 572 control transfers. Because a predictor should affect control-flow speculation but not architectural results, identical checksums across configurations are a system-level check that speculatively fetched instructions are not reaching the register file — which is what a misprediction repair bug would cause.

A directed unit test. The predictor was also tested standalone, outside the CPU, against patterns with known correct answers — always taken, alternating, period-6, period-8 — with a small model of the fetch/decode/ execute timing driving it. This caught a real bug that the full-system test did not: the harness initially failed to drive the decode-stage flush, which left stale state that fired a spurious override on the next branch and dropped the period-6 and period-8 cases to 66 % and 75 %.

The results are also the cleanest illustration of what history buys. The percentages reported by this test are *prediction accuracy* — correct predictions over total branches — not misprediction rates. On the periodic patterns the perceptron and gshare both reach 99–100 % accuracy, while bimodal manages 25 % on period-4, 33 % on period-6, and **0 % on strict alternation**: on a branch that goes taken, not taken, taken, not taken, the bimodal predictor is wrong every single time.

That last figure is worth pausing on, because a 0 in an accuracy column looks like a broken test. It is the correct and expected behaviour. A 2-bit counter starting in weakly-not-taken predicts not-taken, sees taken, and steps to weakly-taken; it then predicts taken, sees not-taken, and steps back. It oscillates between the

two weak states permanently, always one outcome behind, and never gets one right. This is the textbook pathological case for a counter-based predictor, and it is why the test expects bimodal to fail these patterns rather than pass them.

```

=====
predictor unit test -- perceptron
=====
always taken      period 1 : 2000/2000 = 100% (need >= 99%) PASS
always not taken  period 1 : 2000/2000 = 100% (need >= 99%) PASS
alternating TN    period 2 : 2000/2000 = 100% (need >= 99%) PASS
period-4 TTNN     period 4 : 2000/2000 = 100% (need >= 99%) PASS
period-6 TTTNNN   period 6 : 1997/2000 = 99% (need >= 99%) PASS
period-8 TTNTNTN  period 8 : 2000/2000 = 100% (need >= 99%) PASS
=====
ALL CHECKS PASSED
=====

rtl/tb/TB_predictor.sv:219: $finish called at 144018000 (1ps)

=====
predictor unit test -- bimodal
=====
always taken      period 1 : 2000/2000 = 100% (need >= 99%) PASS
always not taken  period 1 : 2000/2000 = 100% (need >= 99%) PASS
alternating TN    period 2 : 0/2000 = 0% (need >= 0%) PASS
period-4 TTNN     period 4 : 500/2000 = 25% (need >= 0%) PASS
period-6 TTTNNN   period 6 : 668/2000 = 33% (need >= 0%) PASS
period-8 TTNTNTN  period 8 : 500/2000 = 25% (need >= 0%) PASS
=====
ALL CHECKS PASSED
=====

rtl/tb/TB_predictor.sv:219: $finish called at 144018000 (1ps)

=====
predictor unit test -- gshare
=====
always taken      period 1 : 2000/2000 = 100% (need >= 99%) PASS
always not taken  period 1 : 2000/2000 = 100% (need >= 99%) PASS
alternating TN    period 2 : 2000/2000 = 100% (need >= 0%) PASS
period-4 TTNN     period 4 : 2000/2000 = 100% (need >= 0%) PASS
period-6 TTTNNN   period 6 : 1993/2000 = 99% (need >= 0%) PASS
period-8 TTNTNTN  period 8 : 2000/2000 = 100% (need >= 0%) PASS
=====
ALL CHECKS PASSED
=====

```

Figure 6: Standalone predictor unit test. Directed patterns with known-correct outcomes, checked automatically rather than by eye.

Two independent simulators. Everything was run under both Icarus Verilog and Vivado’s xsim. They are different compilers with different elaboration semantics, and they agree exactly: 13 571 cycles, 246 mispredictions, 262 overrides, checksum 801. Agreement between two independent toolchains rules out a large class of tool-specific simulation artefacts.

The two tools also disagree usefully. Vivado rejected RTL that Icarus had accepted for weeks — an `always_ff` block writing counters that an `initial` block also wrote, which violates the SystemVerilog single-driver rule. Icarus tolerates the violation; Vivado does not, and the standard is on Vivado’s side.

11 Simulation results

The benchmark is my own: twelve kernels chosen to stress different branch behaviours — always-taken and always-not-taken, strictly alternating, period-3 and period-8 patterns, pseudo-random at 50 % and at 1 %, a deliberately correlated pattern, nested conditionals, short inner loops with data-dependent trip counts, a long loop, and a recursive factorial. It counts itself using the custom CSRs, so the numbers are what the hardware saw.

Two observations from that table beyond the headline.

First, gshare is *worse* than plain bimodal here (13.5 % against 11.5 %). That is not a mistake. Several

```

=====
PREDICTOR COMPARISON (benchmark region, as measured by the program)
=====
              notaken    bimodal    gshare    perceptron
-----
cycles          17902      13696      13842      13571
instructions     12457      12457      12457      12457
control transfers 3572      3572      3572      3572
mispredictions   2515      412      485      246
CPI              1.437      1.099      1.111      1.089
mispred rate %   70.4      11.5      13.5      6.8
-----
speedup vs notaken 1.000x    1.307x    1.293x    1.319x
mispreds vs bimodal +510.4%  +0.0%    +17.7%   -40.3%
-----
[OK] all configurations agree on final sink = 801, 12457 instructions, 3572 control transfers
    -> the predictors are architecturally transparent.

```

Figure 7: All four predictors, same program, same input. The perceptron is the only configuration below 10% misprediction rate. The final line is the correctness check — identical checksum, instruction count and control-transfer count across all four.

kernels have short, strongly-biased branches where mixing global history into the index destroys a pattern the bimodal table had already learned — classic aliasing. It is a useful reminder that “more history” is not monotonically better, and it is the same effect the perceptron avoids by weighting history bits individually rather than hashing them together.

Second, the gap widens with run length. At LOOPS=2000 the rates become 72.1% / 9.1% / 5.1% / **3.4%**. The perceptron has more training data and more opportunity to exploit long correlations, which is what the theory predicts.

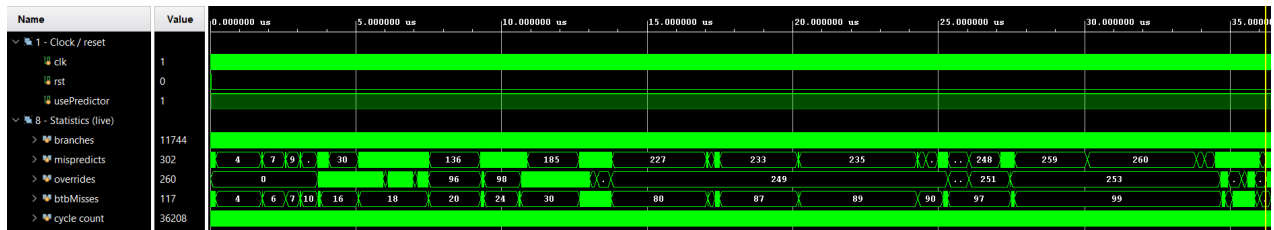


Figure 8: Whole program, roughly 36 μ s on the testbench’s 1 ns timebase. That timebase is a simulation convenience and not a claim about operating frequency — the achievable clock is reported separately from static timing analysis in Section 13.3. The live counters show mispredictions and overrides accumulating; both flatten as the predictor trains, while BTB misses saturate at 117 once every branch site has been allocated.

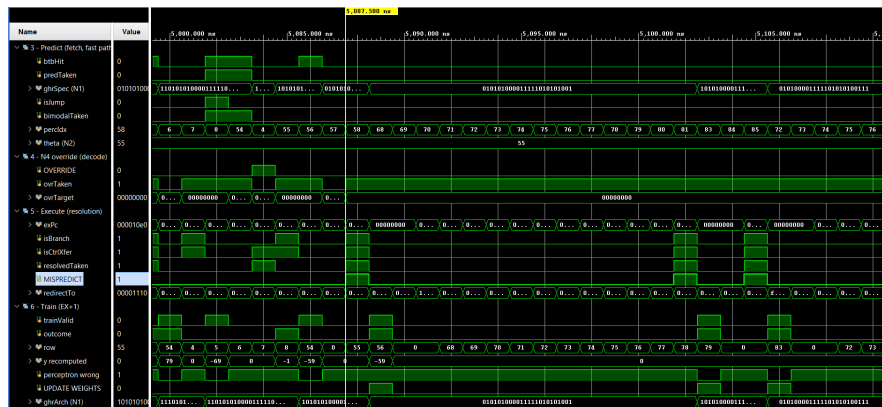


Figure 9: A 30 ns window at 5.08 μ s, with the predictor trained and all four mechanisms visible at once. OVERRIDE pulses at ≈ 5085 ns — the perceptron overruling the fast path (N4). **y** recomputed carries real trained values (+79, -69, -59) rather than the zeros of cold start. θ has adapted from its initial 60 down to 55 (N2). **ghrSpec** and **ghrArch** now hold visibly different values, which is the speculative history running ahead of the committed one (N1). At the cursor, MISPREDICT fires and UPDATE WEIGHTS follows one cycle later.

12 Design-space exploration

Every geometry parameter was swept independently with the rest held at the default, so that each chosen value rests on a measurement rather than an assumption.

=====									
SWEEP	gate	(BP_CONF_GATE)							
CONF_GATE	cycles	mispred	rate%	condMis	percOnly	bimOnly	hybrid	btbMiss	sink
0	13601	257	7.1	323	330	479	330	117	801
1	13599	256	7.1	321	330	479	329	117	801
2	13596	255	7.1	320	330	479	328	117	801
4	13592	254	7.1	318	330	479	326	117	801
8	13582	250	6.9	313	331	479	320	117	801
16	13571	246	6.8	307	332	479	316	117	801
32	13567	246	6.8	307	332	479	315	117	801
64	13693	377	10.5	442	335	481	450	117	801
=====									
best: BP_CONF_GATE=32 at 13567 cycles									

Figure 10: The N3 confidence gate. Width 0 disables the hybrid entirely. The optimum is broad between 16 and 32, and at 64 the predictor collapses to worse than no gate at all.

The gate sweep is the most informative single result in the project. At width 0 — a pure perceptron — the benchmark takes 257 mispredictions. Widening the gate to 16 reduces that to 246. Widening it to 64 sends it to 377, which is *worse than disabling the gate*. The mechanism is straightforward once seen: a perceptron with a small but non-zero margin is still usually right, so a wide gate hands away predictions the perceptron would have got correct. The gate has to be far narrower than the training threshold, and the sweep is what told me where.

=====									
SWEEP	hist	(BP_HIST)							
HIST	cycles	mispred	rate%	condMis	percOnly	bimOnly	hybrid	btbMiss	sink
4	13689	368	10.3	438	406	481	445	117	801
8	13591	271	7.5	339	350	479	344	117	801
12	13566	244	6.8	312	328	479	317	117	801
16	13573	246	6.8	311	327	479	316	117	801
24	13571	246	6.8	307	332	479	316	117	801
32	13569	243	6.8	305	326	479	311	117	801
40	13580	250	6.9	314	335	479	322	117	801
48	13596	261	7.3	325	344	479	334	117	801
=====									
best: BP_HIST=12 at 13566 cycles									

Figure 11: History length. Accuracy saturates at about 12 bits on this benchmark; everything from 12 to 32 is within noise of 6.8%.

Collecting the other three sweeps:

Parameter	Range swept	Knee	Behaviour past the knee
BP_HIST	4 – 48	12	flat to 32, slight regression at 40–48
BP_INDEX_BITS	4 – 9	7	8 and 9 give 244 vs 246 — noise
BP_W_BITS	4 – 10	5	identical from 6 upward
BP_BTBT_ENTRIES	16 – 512	256	512 no better; below 128 it dominates everything

This produced a finding I did not expect and should state plainly: **the configuration I implemented is over-provisioned**. The sweeps show the same accuracy at a third of the storage.

	history	weights	table	status
As built	24 bits	8 bits	25 600 bits	measured — every number in this report
Sweep knee	12 bits	5 bits	≈8 300 bits	not built — same accuracy in simulation only

The smaller configuration has been simulated, not synthesised, so its effect on area and F_{max} is an expectation rather than a result. Every area and timing figure in this report is for the 24-bit, 8-bit design. Since the

predictor’s area is the reason it loses on estimated runtime (Section 14), this is the most consequential result of the sweep, and it emerged after the implementation numbers had been collected.

The BTB sweep is also worth a note: below 128 entries the BTB miss count rises from 117 to 349 and swamps every difference between direction predictors. A good direction predictor cannot compensate for not knowing where the branch goes.

13 FPGA implementation

Target is an Artix-7 xc7a35tcpg236-1 — the part on a Basys 3 board. Synthesis and implementation are run out of context on the core, so that I/O buffer and clock-tree delays, which are identical across all four predictors, do not dilute the comparison.

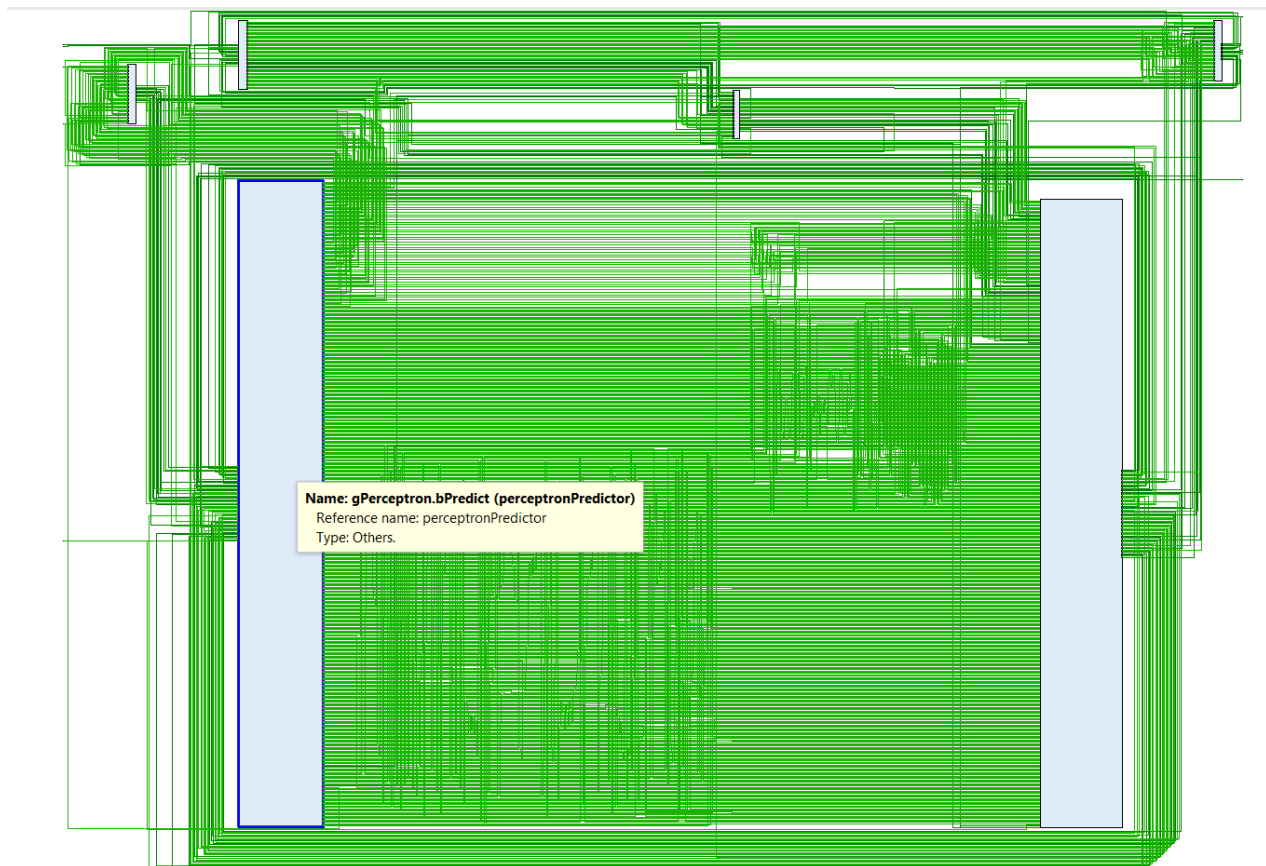


Figure 12: Post-synthesis netlist with the perceptron block highlighted. At this point the weight table has become distributed RAM and the adder tree real carry logic.

13.1 Making it fit

The first synthesis attempt asked for 98 % of the device’s flip-flops and would not fit. The cause was the reset. Resetting every bit of the weight table and the BTB forces the tool to infer flip-flops — 25 600 for the weights plus 15 360 for the BTB. Removing the reset from those arrays lets the same storage infer distributed RAM instead, and power-up state comes from `initial` blocks, which map to the RAM’s initialisation contents on an FPGA and give the same zeroed start in simulation. Roughly 41 000 flip-flops became about 1 280 LUTs.

13.2 Where the area goes

The predictor occupies 2 782 of the core’s 4 412 slice LUTs — 63 % — and 1 472 of its 1 516 LUTRAMs. It is larger than the entire rest of the CPU including the ALU, the register file and all five pipeline stages combined. This is the main reason the predictor loses on estimated runtime, and it is why the over-provisioning result in Section 12 is significant.

13.3 Timing

All four configurations are constrained to a common 10 ns period and none of them meets it. That is the measurement method, not a defect — tightening a constraint until it fails and reading the slack is how

Summary

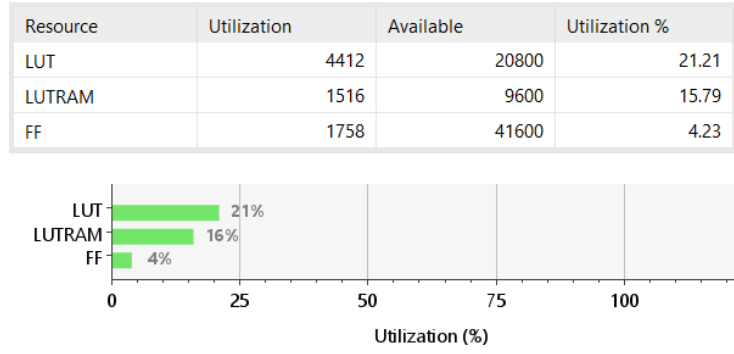


Figure 13: Post-implementation utilisation. 21 % of LUTs, 16 % of distributed RAM, 4 % of flip-flops. It fits comfortably — once the reset question is settled.

Name	^1	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)	LUT as Memory (9600)
core		4412	1758	224	1444	2896	1516
decode (stageDecode)		265	607	64	288	221	44
csrF (csrFile)		160	512	64	223	160	0
regF (registerFile)		63	0	0	27	19	44
execute (stageExecute)		1173	256	48	477	1173	0
alu (alu)		302	0	26	153	302	0
fetch (stageInstFetch)		23	91	0	69	23	0
gPerceptron.bPredict (perceptronPredictor)		2782	516	112	884	1310	1472
mem (stageMem)		67	146	0	98	67	0
wback (stageWback)		103	142	0	92	103	0

Figure 14: Utilisation by hierarchy. The predictor is the largest block in the design by a wide margin.

maximum frequency is found. For the perceptron:

$$F_{max} = \frac{1000}{T - \text{WNS}} = \frac{1000}{10.0 - (-1.620)} = 86.1 \text{ MHz.}$$

The hold violation in the same report (WHS = -0.441 ns) is an out-of-context artefact. Vivado says so itself in the log — with no clock source defined it cannot estimate clock skew, and with zero external I/O delays it manufactures hold failures at the boundary. Setup on internal register-to-register paths, which is what is being reported, is unaffected.

The critical path is the most interesting implementation result, and it is not where I expected it. It runs from a register-file address register, through the forwarding multiplexers, through the ALU's carry chain for the branch comparison, and into the next-PC logic. Exactly one LUT of the perceptron sits on it. **After N4 and pipelined training, the perceptron is no longer the limiting timing path; the remaining critical path is the core's own register-file / forwarding / ALU branch-resolution / next-PC loop.** That is precisely what the overriding organisation was built to achieve. It also means further effort on the predictor's own timing would buy nothing until the core's forwarding path is addressed — the frequency is now limited by the machine, not by the predictor attached to it.

14 The verdict: time to solution

Fewer mispredictions is only a win if the clock survives them, so the figure that decides the comparison is benchmark cycles divided by achievable frequency.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -1.619 ns	Worst Hold Slack (WHS): -0.441 ns	Worst Pulse Width Slack (WPWS): 3.750 ns
Total Negative Slack (TNS): -1324.620 ns	Total Hold Slack (THS): -114.326 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 1621	Number of Failing Endpoints: 1346	Number of Failing Endpoints: 0
Total Number of Endpoints: 16650	Total Number of Endpoints: 16650	Total Number of Endpoints: 3318

Timing constraints are not met.

Figure 15: Timing summary at a 10 ns constraint. The constraint is deliberately not met: all four predictors are implemented against the same 10 ns target and F_{max} is read from the resulting slack.

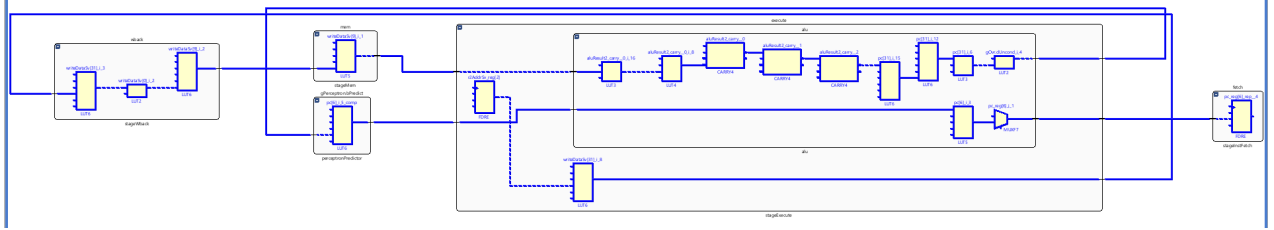


Figure 16: The worst setup path. It starts at a register-file address register in execute, runs through the writeback and memory forwarding logic, through the ALU's carry chain, and ends at the program counter in fetch.

	not-taken	bimodal	gshare	perceptron
LUT as logic	936	1 278	1 337	2 896
LUT as memory	44	940	940	1 516
Slice LUTs (total)	980	2 218	2 277	4 412
Slice registers	1 112	1 184	1 186	1 758
Misprediction rate	70.4 %	11.5 %	13.5 %	6.8 %
Benchmark cycles	17 902	13 696	13 842	13 571
F_{max} (MHz)	92.4	94.3	96.3	86.1
Est. runtime at F_{max} (μs)	193.7	145.2	143.7	157.6

Sources. Area figures are post-place-and-route; F_{max} is from static timing analysis as $1000/(T - \text{WNS})$ at $T = 10$ ns; misprediction rate and cycles are from RTL simulation via the counters of Section 6.1. The last row is derived, not measured:

$$\text{estimated runtime} = \frac{\text{benchmark cycles}}{F_{max}}.$$

No configuration was run on physical hardware, so these are the execution times the measured cycle counts and achieved frequencies imply, not board measurements.

Gshare has the lowest estimated time to solution on this benchmark. The perceptron predicts better than everything else by a clear margin and executes in the fewest cycles, but at the achieved frequencies it still finishes 9.7 % slower, because it gives up 10 MHz to do it.

Three things follow from this that a study based on cycle counts alone would not have shown:

1. **The cycle advantage is real but thin.** The perceptron saves 271 cycles over gshare — 2 % — while the frequency penalty is 11 %. Accuracy differences have to be large to pay for frequency.
2. **The benchmark is too short to favour it.** At L0OPS=2000 the accuracy gap widens from 13.5/6.8 to 5.1/3.4. The perceptron's advantage grows with run length; a 36 μ s benchmark does not give it room.
3. **The design is over-provisioned** by roughly 3 $\times$ in storage (Section 12). A 12-bit-history, 5-bit-weight version should recover part of the frequency gap at the same accuracy. This version has not been built, so the figure is an expectation rather than a result.

Both figures carry ± 5 % place-and-route noise, measured by running each implementation twice against different targets. The 143.7 vs 157.6 gap is outside that; the 145.2 vs 143.7 gap between bimodal and gshare is not, and I would not claim gshare beats bimodal on this evidence.

15 Limitations

- **One benchmark.** Twelve kernels I wrote myself, chosen to exercise branch behaviour. They are not Dhrystone or CoreMark and they are not a real workload. The correct next step is Embench or larger traces.
- **No board deployment.** The RTL has been synthesised and placed-and-routed for the Artix-7 target, and all area and timing figures come from those runs, but a bitstream was not generated and the design has not been run on a Basys 3. The remaining work is understood and is mostly memory restructuring: the instruction memory currently has an asynchronous read, which block RAM cannot provide, and the data memory reads four independently computed byte addresses to support unaligned access, which needs restructuring into byte lanes with byte enables. A clocking wizard and a real UART would also be required.
- **Out-of-context synthesis.** Running the core out of context keeps I/O and clock-tree delays — identical across all four predictors — from diluting the comparison, but it also means the reported hold analysis is not meaningful (Section 13.3).
- **The instruction counter is a proxy.** As described in Section 6.1, 0x82 counts execute-stage cycles with a non-zero PC rather than signalling architectural retirement. The two coincide for this pipeline, but the counter would need reworking for a machine with exceptions.
- **The over-provisioning is unresolved.** The 12-bit, 5-bit configuration has been simulated but not implemented.
- **No aliasing study.** 128 perceptron rows for a benchmark with 117 distinct branch sites means aliasing is nearly absent here. On real code with thousands of branch sites it would not be, and that is exactly where perceptron predictors are usually reported to struggle.

16 Conclusions and further work

The perceptron predictor works as published and is worth its accuracy claim: on this benchmark it mispredicts 6.8% of control transfers where the best conventional predictor I implemented manages 11.5%, and the margin widens as the program runs longer. The overriding organisation does what it was designed to do — after N4 the perceptron is no longer on the critical path, which moved to the core’s own branch-resolution loop.

It still loses. At 63% of the core’s LUTs it costs 10 MHz, and 10 MHz is more than a two-percent cycle advantage can repay. The conclusion I would defend is not that perceptron prediction is a bad idea, but that predictor accuracy is not by itself a performance result: it has to be evaluated together with the frequency and area it costs the machine it sits in, and a study that reports only misprediction rates cannot tell which predictor is faster.

In priority order, the work that would most change these numbers:

1. **Implement and measure the 12-bit-history, 5-bit-weight configuration.** It is the only change expected to recover frequency at no accuracy cost, and it is a configuration change rather than a redesign.
2. **Restructure the memories for block RAM** and build a bitstream, so that estimated runtimes can be replaced by measured ones.
3. **Extend the benchmark** to Embench or captured traces, which would also test the predictor under real aliasing pressure.
4. **Attack the core’s critical path.** Since the predictor is no longer the limiter, the next frequency gain has to come from the register-file-to-ALU forwarding path, not from the predictor.

17 Tooling

RTL is SystemVerilog. Software is compiled with `riscv-none-elf-gcc` 15.2 against a custom linker script and boot sequence. Simulation is Icarus Verilog 12 and Vivado `xsim` 2023.2; synthesis and implementation are Vivado 2023.2. The build, the four-way comparison, the parameter sweeps and the unit test are driven from a Makefile, so every number in this document regenerates with a single command. Waveform layout for `xsim` is scripted as well — nine pre-grouped signal sets with appropriate radices — so the figures here are reproducible rather than hand-arranged.

References

- [1] D. A. Jiménez and C. Lin. “Dynamic Branch Prediction with Perceptrons.” *HPCA-7*, 2001.
- [2] D. A. Jiménez, S. W. Keckler and C. Lin. “The Impact of Delay on the Design of Branch Predictors.” *MICRO-33*, 2000.
- [3] S. McFarling. “Combining Branch Predictors.” WRL Technical Note TN-36, DEC, 1993.
- [4] A. Sez nec. “Analysis of the O-GEometric History Length Branch Predictor.” *ISCA-32*, 2005.